Maschinelles Sehen

Projektbericht: Echtzeit-Objekterkennung von Alltagsgegenständen mit YOLO und Transfer Learning

BHT Berliner Hochschule für Technik

Eingereicht von

Mazen Younis – 952106
Mert Satici – 951498

Fachbereich VI – Technische Informatik – Embedded Systems (M. Eng.)
Berliner Hochschule für Technik

Abgabedatum: 28.05.2026

Studiere Zukunft

Betreuer
Prof. Dr. Kristian Hildebrand Berliner Hochschule für Technik

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Projektziel und Vorgehen 1
1.2 Abgrenzung 1

2 Grundlagen **1**
2.1 Objekterkennung und YOLO 1
2.2 Transfer Learning und Annotation 2
2.3 Evaluationsmetriken 2

3 Datensatz und Annotation **2**
3.1 Bildsammlung und Zielklassen 2
3.2 Annotation und Datensatzaufteilung 3
3.3 Preprocessing und Export 3

4 Methodik und Implementierung **4**
4.1 Projektstruktur und Entwicklungsumgebung 4
4.2 Baseline und Datensatzeinbindung 4
4.3 Fine-Tuning und Evaluation 5

5 Training und Evaluation **5**
5.1 Trainingsaufbau 5
5.2 Validierungs- und Testergebnisse 5
5.3 Klassenspezifische Ergebnisse 6

6 Ergebnisse und Diskussion **6**
6.1 Vergleich mit dem Baseline-Modell 6
6.2 Diskussion der Ergebnisse 8
6.3 Grenzen des Modells 8

7 Fazit und Ausblick **8**

1 Einleitung

Die automatische Erkennung von Objekten in Bildern ist ein zentraler Bereich des maschinellen Sehens. Im Gegensatz zur reinen Bildklassifikation soll bei der Objekterkennung nicht nur bestimmt werden, welche Objekte in einem Bild vorkommen, sondern auch, wo sie sich befinden.

In diesem Projekt wird ein Objekterkennungssystem für alltägliche Gegenstände auf einem Schreibtisch umgesetzt. Betrachtet werden fünf Objektklassen: `bottle`, `cell_phone`, `cup`, `laptop` und `mouse`. Ziel ist es, ein vortrainiertes YOLOv8n-Modell zunächst als Baseline zu testen und anschließend mit einem eigenen Datensatz feinzujustieren.

1.1 Projektziel und Vorgehen

Das Projekt untersucht, ob sich die Erkennungsleistung für eine spezifische Schreibtischumgebung durch Transfer Learning verbessern lässt. Dafür wurde ein eigener Bilddatensatz erstellt, in Roboflow annotiert und im YOLOv8-Format exportiert. Anschließend wurde das vortrainierte Modell mit diesem Datensatz weitertrainiert und anhand von Validierungs- und Testdaten bewertet.

Die wichtigsten Arbeitsschritte sind:

- Sammlung und Annotation eigener Schreibtischbilder
- Baseline-Test mit einem vortrainierten YOLOv8n-Modell
- Fine-Tuning des Modells auf fünf Zielklassen
- Evaluation und Vergleich der Ergebnisse

1.2 Abgrenzung

Das Projekt beschränkt sich auf die praktische Anwendung von Transfer Learning mit YOLOv8. Es wird keine neue Modellarchitektur entwickelt. Andere sichtbare Objekte wie Tastatur, Monitor, Kabel oder Tischfläche werden nicht als Zielklassen betrachtet. Der Fokus liegt ausschließlich auf den fünf definierten Schreibtischobjekten.

2 Grundlagen

Dieses Kapitel fasst die theoretischen Grundlagen zusammen, die für die Umsetzung und Bewertung des Projekts notwendig sind.

2.1 Objekterkennung und YOLO

Maschinelles Sehen beschäftigt sich mit der automatischen Analyse von Bildern oder Videodaten. Bei der Bildklassifikation wird einem gesamten Bild eine Klasse zugeordnet. Bei der Objekterkennung wird zusätzlich bestimmt, wo sich einzelne Objekte im Bild befinden. Dafür werden Bounding Boxes verwendet, also rechteckige Rahmen um die erkannten Objekte.

YOLO steht für „You Only Look Once“ und ist eine Modellfamilie für die Objekterkennung. Das Modell verarbeitet ein Bild in einem Durchlauf und sagt dabei Objektklassen, Bounding Boxes und Konfidenzwerte voraus. Dadurch ist YOLO effizient und eignet sich gut für praktische Objekterkennungsaufgaben.

In diesem Projekt wurde YOLOv8n verwendet. Das „n“ steht für „nano“ und bezeichnet eine kleine Modellvariante. Sie wurde gewählt, weil sie schneller trainiert und für ein Projekt mit begrenztem Datensatz und lokaler Entwicklungsumgebung gut geeignet ist.

2.2 Transfer Learning und Annotation

Ein vortrainiertes Modell wurde bereits auf einem großen allgemeinen Datensatz trainiert und besitzt dadurch grundlegendes visuelles Wissen über Formen, Kanten und Objektstrukturen. Beim Transfer Learning wird dieses Modell nicht von Grund auf neu trainiert, sondern mit einem eigenen Datensatz weitertrainiert. Dieser Schritt wird als Fine-Tuning bezeichnet.

Für das Fine-Tuning werden annotierte Bilder benötigt. Jede Annotation enthält ein Klassenlabel und eine Bounding Box für das jeweilige Objekt. In diesem Projekt wurden die Zielobjekte manuell annotiert, damit das YOLOv8n-Modell gezielt auf die fünf Klassen `bottle`, `cell_phone`, `cup`, `laptop` und `mouse` angepasst werden konnte.

2.3 Evaluationsmetriken

Zur Bewertung des Modells wurden Precision, Recall, mAP50 und mAP50-95 verwendet. **Precision** beschreibt, wie viele erkannte Objekte tatsächlich korrekt sind. **Recall** beschreibt, wie viele der tatsächlich vorhandenen Objekte gefunden wurden.

mAP50 steht für mean Average Precision bei einer Überlappungsschwelle von 50 Prozent. Eine Vorhersage gilt dabei als korrekt, wenn die vorhergesagte Bounding Box ausreichend mit der tatsächlichen Bounding Box überlappt. **mAP50-95** ist strenger, da die Leistung über mehrere Überlappungsschwellen von 50 bis 95 Prozent gemittelt wird.

3 Datensatz und Annotation

Dieses Kapitel beschreibt die Erstellung des eigenen Datensatzes, einschließlich Bildsammlung, Annotation, Aufteilung und Export im YOLOv8-Format.

3.1 Bildsammlung und Zielklassen

Für das Projekt wurde ein eigener Bilddatensatz mit realen Schreibtischszenen erstellt. Die Bilder zeigen typische Arbeitsumgebungen mit mehreren Alltagsgegenständen auf oder in der Nähe eines Schreibtisches. Der Datensatz besteht aus 80 Bildern, davon 48 eigene Aufnahmen und 32 weitere Aufnahmen eines zweiten Projektmitglieds.

Tab. 1: Übersicht der Bildsammlung

Quelle	Anzahl der Bilder
Eigene Bildsammlung	48
Zweite Bildsammlung	32
Gesamt	80

Für die Objekterkennung wurden fünf Zielklassen definiert: `bottle`, `cell_phone`, `cup`, `laptop` und `mouse`. Andere sichtbare Objekte wie Tastatur, Monitor, Kabel oder Tischfläche wurden nicht annotiert, da sie nicht Teil der Aufgabenstellung sind.

3.2 Annotation und Datensatzaufteilung

Die Annotation wurde in Roboflow durchgeführt. Dafür wurden alle 80 Bilder hochgeladen und die Zielobjekte manuell mit Bounding Boxes markiert. Jede Bounding Box wurde einer der fünf definierten Klassen zugeordnet. Nicht relevante Objekte wurden bewusst nicht markiert.

Nach der Annotation wurde der Datensatz in Trainings-, Validierungs- und Testdaten aufgeteilt. Die Trainingsdaten wurden für das Fine-Tuning verwendet, die Validierungsdaten für die Bewertung während des Trainings und die Testdaten für die abschließende Evaluation auf unbekannten Bildern.

Tab. 2: Aufteilung des Datensatzes

Datensatzteil	Anzahl der Bilder
Training	56
Validierung	16
Test	8
Gesamt	80

Die Aufteilung entspricht ungefähr einem Verhältnis von 70 % Training, 20 % Validierung und 10 % Test.

3.3 Preprocessing und Export

Vor dem Export wurden die Bilder in Roboflow automatisch ausgerichtet und auf eine einheitliche Größe von 640 x 640 Pixeln angepasst. Es wurden keine zusätzlichen Augmentierungen verwendet, damit die erste Trainingsversion einfach und nachvollziehbar bleibt.

Der Datensatz wurde anschließend im YOLOv8-Format exportiert und lokal im Ordner `data/processed/desk_object_detection_v1/` gespeichert. Die Datei `data.yaml` enthält die Pfade zu Trainings-, Validierungs- und Testdaten sowie die Namen der fünf Klassen.

Da der Datensatz mit 80 Bildern relativ klein ist, sind die Ergebnisse als erste Modellversion zu verstehen. Für eine spätere Verbesserung wären zusätzliche Bilder sinnvoll, besonders für kleinere oder schwer erkennbare Objekte wie `cell_phone` und `mouse`.

4 Methodik und Implementierung

Dieses Kapitel beschreibt den praktischen Aufbau des Projekts, einschließlich Projektstruktur, Entwicklungsumgebung, Baseline-Test, Fine-Tuning und Evaluation.

4.1 Projektstruktur und Entwicklungsumgebung

Für die Umsetzung wurde eine einfache Projektstruktur angelegt, um Bilddaten, Quellcode, Ergebnisse und Dokumentation getrennt zu organisieren. Die wichtigsten Ordner sind:

```
desk-object-detection-yolo/
data/
docs/
results/
src/
README.md
requirements.txt
.gitignore
```

Listing 1: Vereinfachte Projektstruktur

Der Ordner `data/` enthält die Rohbilder, Beispielbilder und den aus Roboflow exportierten YOLOv8-Datensatz. In `src/` befinden sich die Python-Skripte für Baseline-Erkennung, Training, Evaluation und Modellvergleich. Die Ergebnisse wurden unter `results/` gespeichert.

Das Projekt wurde lokal in PyCharm mit Python umgesetzt. Für die Objekterkennung und das Training wurden Ultralytics YOLOv8, PyTorch, OpenCV, NumPy und Matplotlib verwendet. Die Abhängigkeiten wurden in `requirements.txt` dokumentiert.

4.2 Baseline und Datensatzeinbindung

Zu Beginn wurde ein vortrainiertes YOLOv8n-Modell ohne eigenes Training getestet. Dieser Baseline-Test diente dazu, die Leistung eines allgemeinen Modells mit den späteren Ergebnissen des feinjustierten Modells zu vergleichen. Dafür wurden fünf Beispielbilder aus `data/test_images/` mit dem Skript `src/detect_baseline.py` ausgewertet.

Der Baseline-Test zeigte, dass das Modell einige relevante Objekte erkannte, aber auch nicht relevante Klassen wie `keyboard` oder `dining table` ausgab. Danach wurde der in Roboflow annotierte Datensatz im YOLOv8-Format exportiert und lokal eingebunden. Die Datei `data.yaml` enthält die Pfade zu Trainings-, Validierungs- und Testdaten sowie die fünf Klassen.

```
train: train/images
val: valid/images
test: test/images

nc: 5
names: ['bottle', 'cell_phone', 'cup', 'laptop', 'mouse']
```

Listing 2: Ausschnitt aus der verwendeten data.yaml

4.3 Fine-Tuning und Evaluation

Für das Fine-Tuning wurde das vortrainierte Modell `yolov8n.pt` als Ausgangspunkt verwendet. Das Training wurde mit `src/train_yolo.py` durchgeführt. Die wichtigsten Trainingsparameter sind in Tabelle 3 dargestellt.

Tab. 3: Trainingsparameter

Parameter	**Wert**
Modell	YOLOv8n
Vortrainierte Gewichte	`yolov8n.pt`
Epochen	50
Bildgröße	640
Batch-Größe	8
Trainingsgerät	CPU, Apple M4

Das beste trainierte Modell wurde als `best.pt` gespeichert. Anschließend wurde es mit `src/evaluate_model.py` auf dem Testdatensatz bewertet. Zusätzlich wurde mit `src/compare_models.py` ein direkter Vergleich zwischen dem vortrainierten und dem feinjustierten Modell auf denselben fünf Beispielbildern durchgeführt.

5 Training und Evaluation

Dieses Kapitel fasst das Fine-Tuning des YOLOv8n-Modells sowie die wichtigsten Validierungs- und Testergebnisse zusammen.

5.1 Trainingsaufbau

Das Fine-Tuning wurde mit dem vortrainierten Modell `yolov8n.pt` durchgeführt. Als Grundlage diente der eigene Datensatz mit 80 annotierten Bildern, aufgeteilt in 56 Trainingsbilder, 16 Validierungsbilder und 8 Testbilder.

Das Modell wurde für 50 Epochen mit einer Bildgröße von 640 und einer Batch-Größe von 8 trainiert. Das Training wurde lokal auf einem Apple-M4-System mit CPU durchgeführt. Das beste Modell wurde anschließend als `best.pt` gespeichert und für die Evaluation verwendet.

5.2 Validierungs- und Testergebnisse

Nach dem Training wurde das Modell zunächst auf dem Validierungsdatensatz und anschließend auf dem separaten Testdatensatz bewertet. Die wichtigsten Ergebnisse sind in Tabelle 4 zusammengefasst.

Tab. 4: Gesamtergebnisse auf Validierungs- und Testdatensatz

Datensatz	Precision	Recall	mAP50	mAP50-95
Validierung	0,839	0,811	0,891	0,797
Test	0,853	0,791	0,903	0,800

Die Ergebnisse zeigen, dass das Modell nach dem Fine-Tuning eine gute Gesamtleistung erreicht. Der mAP50-Wert von 0,903 auf dem Testdatensatz zeigt, dass die meisten Objekte grundsätzlich korrekt erkannt wurden. Der etwas niedrigere Recall zeigt jedoch, dass einzelne vorhandene Objekte noch übersehen wurden.

5.3 Klassenspezifische Ergebnisse

Die Leistung unterschied sich je nach Objektklasse. `bottle`, `cup` und `laptop` wurden vergleichsweise zuverlässig erkannt. Schwieriger waren `cell_phone` und `mouse`, da diese Objekte häufig klein, dunkel, reflektierend oder teilweise verdeckt waren.

Tab. 5: Klassenspezifische Ergebnisse auf dem Testdatensatz

Klasse	Precision	Recall	mAP50	mAP50-95
`bottle`	1,000	0,784	0,995	0,821
`cell_phone`	0,771	0,571	0,766	0,672
`cup`	0,965	1,000	0,995	0,895
`laptop`	0,661	1,000	0,995	0,932
`mouse`	0,866	0,600	0,765	0,677

Insgesamt war das Fine-Tuning erfolgreich, da das Modell gute Gesamtwerte erreichte und auf die fünf Zielklassen spezialisiert wurde. Gleichzeitig zeigen die klassenspezifischen Ergebnisse, dass besonders für `cell_phone` und `mouse` zusätzliche Trainingsbilder sinnvoll wären.

6 Ergebnisse und Diskussion

Dieses Kapitel fasst die wichtigsten Ergebnisse zusammen und diskutiert die Unterschiede zwischen dem vortrainierten und dem feinjustierten Modell.

6.1 Vergleich mit dem Baseline-Modell

Zu Beginn wurde ein vortrainiertes YOLOv8n-Modell auf fünf Beispielbildern getestet. Dieses Modell erkannte einige relevante Objekte, gab jedoch auch nicht relevante Klassen wie `keyboard` oder `dining table` aus. Nach dem Fine-Tuning wurde das trainierte Modell auf denselben Bildern getestet. Dabei zeigte sich, dass es stärker auf die fünf Zielklassen spezialisiert war.

Tab. 6: Vergleich zwischen Baseline-Modell und feinjustiertem Modell

Aspekt	**Vortrainiertes Modell**	**Feinjustiertes Modell**
Klassenumfang	Allgemeine COCO-Klassen	Nur fünf Projektklassen
Irrelevante Klassen	Erkannte zusätzlich z. B. `keyboard` und `dining table`	Keine irrelevanten Klassen im Vergleichsbild
Anpassung	Nicht auf Schreibtischszenen spezialisiert	Besser an die gewählte Aufgabe angepasst
Typische Fehler	Erkennung nicht relevanter Objekte	Teilweise Mehrfacherkennung von `laptop`

Der Vergleich zeigt, dass das Fine-Tuning die Ausgaben des Modells stärker auf die konkrete Aufgabenstellung eingeschränkt hat. Das feinjustierte Modell erkannte nur noch die definierten Klassen `bottle`, `cell_phone`, `cup`, `laptop` und `mouse`.

Abbildung 1 zeigt eine Beispielvorhersage des feinjustierten Modells. Es erkennt `mouse`, `cup` und `cell_phone` mit Bounding Boxes und Konfidenzwerten.

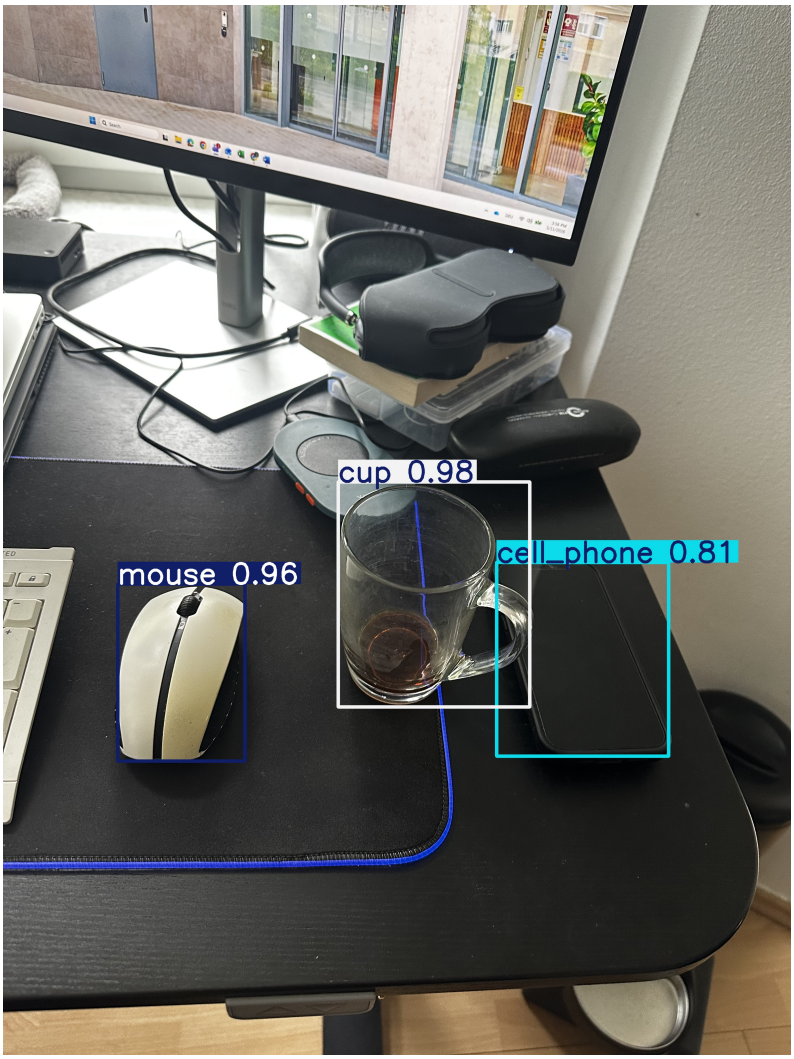


Abb. 1: Beispielhafte Vorhersage des feinjustierten YOLOv8n-Modells auf einem Schreibtischbild

6.2 Diskussion der Ergebnisse

Die Ergebnisse zeigen, dass Transfer Learning für diese Aufgabenstellung sinnvoll war. Das vortrainierte YOLOv8n-Modell hatte bereits allgemeines visuelles Wissen, musste jedoch an die spezifischen Zielklassen angepasst werden. Durch das Fine-Tuning wurde das Modell besser auf die eigene Schreibtischumgebung spezialisiert.

Besonders gute Ergebnisse wurden bei `bottle`, `cup` und `laptop` erreicht. Schwieriger waren `cell_phone` und `mouse`, da diese Objekte häufig klein, dunkel, reflektierend oder teilweise verdeckt waren. Außerdem kam es in einzelnen Bildern zu Mehrfacherkennungen bei `laptop`, vermutlich weil rechteckige Bereiche auf dem Schreibtisch ähnlich aussehen.

6.3 Grenzen des Modells

Das Projektziel wurde grundsätzlich erreicht, da ein vollständiger Workflow von Datensammlung und Annotation bis Fine-Tuning, Evaluation und Modellvergleich umgesetzt wurde. Gleichzeitig ist das aktuelle Modell als erste Version zu verstehen.

Die wichtigsten Grenzen sind:

- Der Datensatz ist mit 80 Bildern relativ klein.
- Einige Klassen sind schwieriger zu erkennen als andere.
- Kleine oder dunkle Objekte werden teilweise übersehen.
- Rechteckige Formen können mit Laptops verwechselt werden.

Für stabilere Ergebnisse wären zusätzliche Trainingsbilder mit mehr Perspektiven, Lichtverhältnissen, Hintergründen und Objektpositionen sinnvoll.

7 Fazit und Ausblick

Dieses Kapitel fasst die wichtigsten Erkenntnisse des Projekts zusammen und nennt mögliche Verbesserungen für eine zukünftige Version.

In diesem Projekt wurde ein Objekterkennungssystem für alltägliche Gegenstände auf einem Schreibtisch umgesetzt. Dafür wurde zunächst ein vortrainiertes YOLOv8n-Modell als Baseline getestet. Anschließend wurde ein eigener Datensatz mit 80 Bildern erstellt, in Roboflow annotiert und im YOLOv8-Format exportiert.

Das Modell wurde mithilfe von Transfer Learning auf die fünf Zielklassen `bottle`, `cell_phone`, `cup`, `laptop` und `mouse` feinjustiert. Die Evaluation zeigte, dass das trainierte Modell grundsätzlich gute Ergebnisse erreicht. Auf dem Testdatensatz wurde ein mAP50-Wert von 0,903 erzielt.

Der Vergleich mit dem Baseline-Modell zeigte, dass das feinjustierte Modell besser zur Aufgabenstellung passt. Während das vortrainierte Modell auch nicht relevante Klassen wie `keyboard` oder `dining table` erkannte, beschränkte sich das feinjustierte Modell auf die fünf definierten Projektklassen.

Trotz der positiven Ergebnisse ist das Modell noch nicht vollständig robust. Besonders `cell_phone` und `mouse` waren schwieriger zu erkennen. Außerdem kam es teilweise zu Mehrfacherkennungen bei `laptop`. Eine wichtige Einschränkung ist außerdem die geringe Größe des Datensatzes.

Für eine zukünftige Version sollte der Datensatz erweitert werden, besonders mit mehr Bildern von kleinen oder schwierigen Objekten. Zusätzlich könnten Datenaugmentierung, größere YOLOv8-Modelle und ein Test mit Webcam oder Videostream untersucht werden.

Insgesamt zeigt das Projekt, dass Transfer Learning mit YOLOv8 bereits mit einem kleinen eigenen Datensatz zu brauchbaren Ergebnissen führen kann. Für eine stabile Anwendung in unterschiedlichen realen Umgebungen wären jedoch weitere Daten und zusätzliche Trainingsläufe notwendig.